Executable Evidence Management for CAD-Guided Aircraft Inspection

Chuansu Peng[a], Haoran Dong[a], Ruoxuan Wu[a], Peipei Zeng[b]

[a]*School of Aeronautical Engineering, Civil Aviation University of China, China*
[b]*Engineering Training Center, Civil Aviation University of China, China*

Abstract

Engineering changes alter the applicability of evidence supporting an inspection output. We develop executable evidence contracts that connect typed task requirements, versioned numerical producers, calibrated event bounds and source-scoped execution history. Requirement containment reuses a stronger statistical statement; dependency projections retain compatible producer functions; exact-event fallback preserves directly supportable requests. We establish support preservation for aligned calibration inputs and a common confidence allocation, and compare the contract with whole renewal, a conventional dependency cache and an independent rule implementation. An aircraft-skin simulation executes 3,072 producer outputs from 512 independently sampled scenes with varied fastener layouts, missing sites, occlusion and CAD discrepancy. All complete methods support 41 of 48 contexts. The contract reduces evidence-record evaluations from 10,752 to 6,656 (38.10%) while recovering three contexts lost by anchor-only reuse. In 640 paired source-renewal cases, dependency caching reduces measured CPU from 183.99 to 80.49 s; this numerical benefit is shared by the contract and rules implementations. Component regression detects 86 activated hidden couplings on 64 independent inputs, and image-output traces verify persistent attachment checks through change and restoration. A separate 128-scene stress study exposes the operating envelope. The resulting interface makes evidence retention, renewal and issuance explicit and auditable across an inspection output's lifecycle.

Keywords: Executable operating contracts, Evidence reuse, Engineering change management, Risk calibration, Aircraft inspection

1. Introduction

An inspection result is useful when an engineer can identify the aircraft location it describes and the conditions under which its quality statement remains valid. In a CAD-guided workflow, image acquisition, feature extraction and pose estimation jointly determine this information. During operation, requirements and components change independently: a consumer may relax a mapping tolerance, a camera may change exposure, or a decoder may change its extraction rule. The system must decide which evidence can accompany the new output.

This decision has three parts. The requested event must be covered by the statistical statement; the quantities entering that event must come from a compatible producer; and the statement must attach to the numerical version actually used by the output. A dependency graph addresses recomputation, while an evidence decision also depends on event semantics and execution history. These relations are readily separated in software and easily confused in an inspection report.

Engineering knowledge models connect requirements and change dependencies [1, 2, 3]. Incremental computation uses dependency structure to reuse unchanged values [4]. Risk-controlled selection supplies statistical statements for specified programs [5], and runtime assurance checks conditions for permissible execution [6]. We connect these operations through an executable evidence contract: a typed request identifies the event, a producer projection identifies the functions it uses, and a persistent execution record identifies the output receiving the statement.

The contribution is a compositional evidence interface for engineering change. It adds statistical event containment and source-scoped issuance to a conventional dependency cache. An exact-event fallback

preserves directly supportable requests, and an explicit support-preservation result states the conditions for that equivalence. The implementation is evaluated against whole renewal, dependency-aware exact caching and an independent rule implementation with the same semantics. This comparison separates numerical caching benefits from the additional reduction in evidence construction.

The aircraft-skin instance couples pixel extraction and CAD association to the complete evidence lifecycle. Independently sampled layouts, missing fasteners, partial occlusion and CAD discrepancies exercise the image producer. Measured stage execution quantifies renewal cost, paired component updates test hidden coupling, and source traces test withdrawal and restoration on the resulting numerical records. Figure 1 provides the engineering context; Fig. 2 summarizes the architecture.

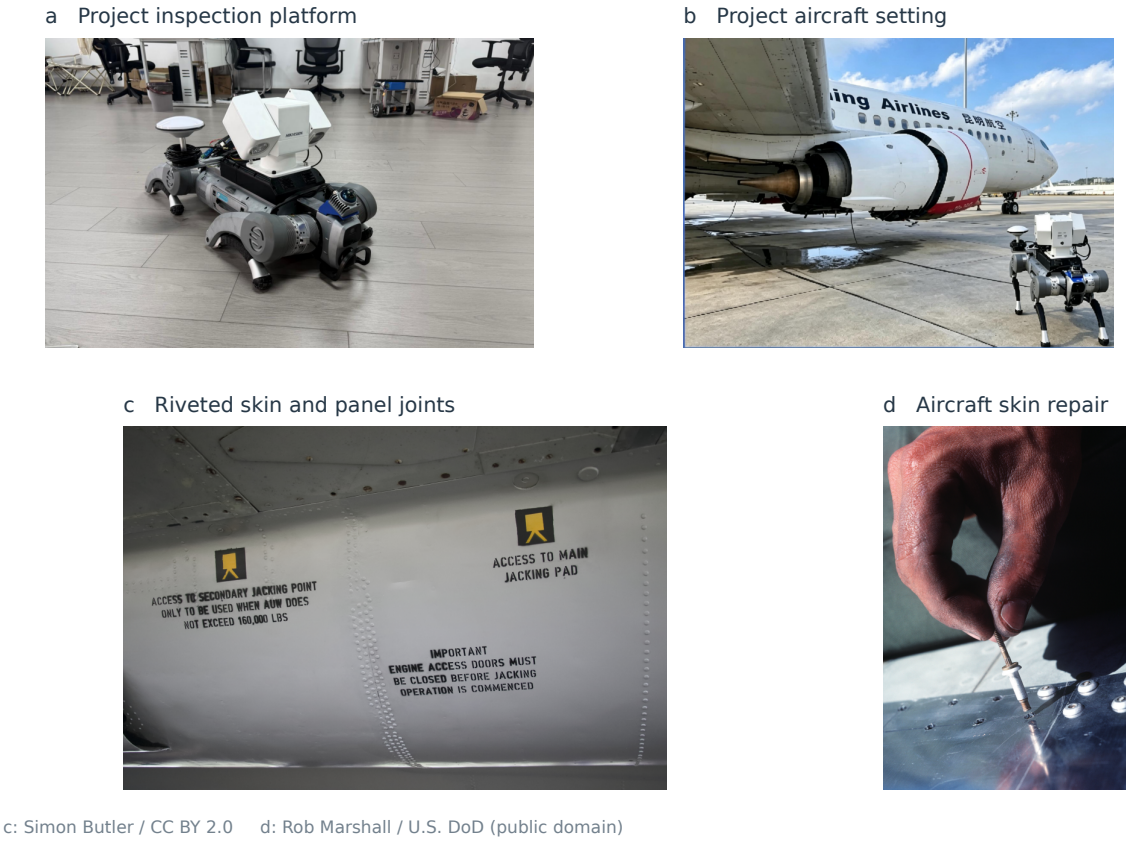


Figure 1: Engineering context at platform and surface scales. (a,b) Original project photographs. (c) Riveted under-fuselage panels on Avro Vulcan XM655, photographed by Simon Butler (CC BY 2.0). (d) Aircraft sheet-metal repair, photographed by Rob Marshall (U.S. Department of Defense, public domain). Full sources and image credits are provided in the Supplementary Material.

2. Related work

2.1. Knowledge representation and engineering change

Process and decision knowledge graphs represent engineering choices and their supporting relations [1, 2]. Semantic change propagation identifies affected components [3, 7]; inspection ontologies connect operating conditions to robot requirements [8]. Representational completeness [9] and executable model conversion [10] motivate evaluating whether a representation supports a concrete decision. Here that decision is evidence applicability after a requirement, producer or execution change.

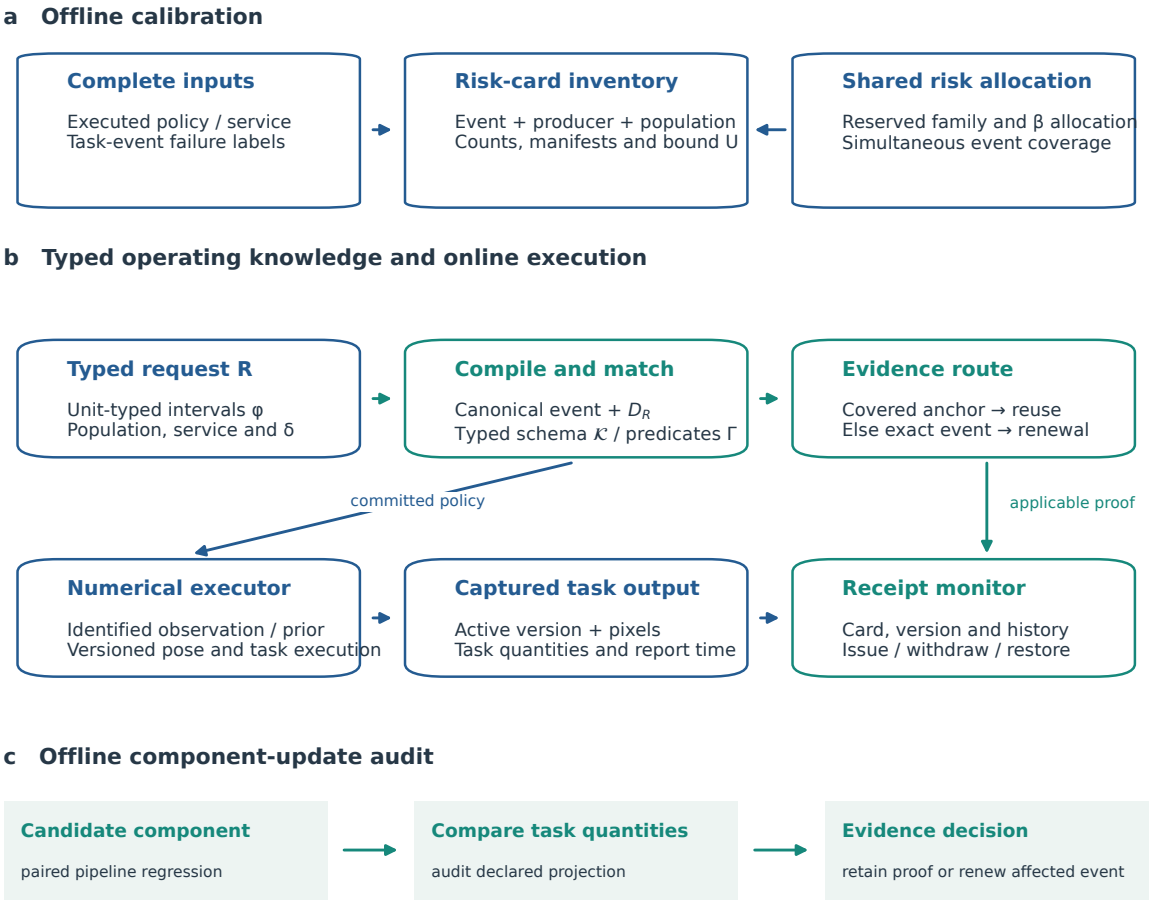


Figure 2: Executable evidence-management architecture. Calibration records establish event bounds; typed requests select compatible evidence and a producer plan. Numerical versions and execution history govern attachment. Component updates pass through regression and dependency checks before their evidence is retained or renewed.

2.2. Incremental computation and numerical services

Dependency-directed computation retains values whose inputs and functions remain unchanged [4]. We use this established mechanism as the numerical cache and as a strong baseline. Pose estimation supplies the transformation used by a CAD-guided inspection output [11, 12]; uncertainty analysis distinguishes fitted residuals from transformation uncertainty [13, 14, 15]. The estimator in this study is an interchangeable numerical provider. The evaluated evidence operation is whether a quality statement remains applicable to its output.

2.3. Risk calibration and runtime contracts

Assumption–guarantee contracts express component obligations [16]. Binomial inversion and risk-controlled selection give finite-family guarantees for specified programs [17, 5, 18]. Runtime assurance connects monitored conditions to execution [6]. Our interface composes event containment, requested-function identity and source history. A rule engine supplied with the same predicates can implement the same semantics; we include that implementation to distinguish the evidence rules from their software representation.

Table 1: Comparison scope and attribution. All complete methods use the same numerical producer and confidence family.

Method	Numerical reuse	Evidence reuse	Role
Whole renewal	Recompute full producer	Exact events	Reference work and support
DAG exact cache	Reuse unchanged stages	Identical events	Incremental-computation baseline
Contract + fallback	Same DAG	Containment, then exact	Effect of evidence semantics
Rules + containment	Same DAG	Same semantics, separate rules	Representation equivalence

3. Executable task and evidence model

3.1. Typed representation and compilation

The operating knowledge structure is

$$\mathcal{K} = (\mathcal{V}, \tau, \mathcal{R}, \sigma, \Gamma), \tag{1}$$

where entities have types τ, typed relations $\mathcal{R}$, immutable specifications σ and executable predicates Γ. The relation signatures include requires : Request $\to$ Event, bounds : Card $\to$ Event, dependsOn : Quantity $\to$ Provider and uses : Receipt $\to$ SourceVersion. Plans select a numerical producer and a supporting card. Type checks reject a relation whose endpoints violate its signature. A source-revision relation requires the same source key and a strictly increasing version sequence. The schema connects an engineering requirement to the evidence and execution information needed to decide its applicability.

A request $R = (g, a, \varphi, \delta)$ fixes population, service, a conjunction of quantity intervals and a risk allowance. Its grammar is

$$\varphi ::= \mathrm{Interval}(m, u, \ell, h) \mid \varphi \wedge \varphi. \tag{2}$$

The compiler checks engineering dimensions, converts units, intersects repeated intervals and produces a canonical event identity. Omitted bounds give unbounded sides. Nonfinite required quantities fail the event. Compilation also aggregates the providers used by the requested quantities. The provider manifest declares the dependencies of implemented quantities. These declarations are engineering interfaces, rather than automatically inferred causal relations. Each execution stage also has a content-addressed cache key containing its registered stage identity, direct parameters and parent keys:

$$h_v = \mathrm{Hash}\big(v, \sigma_v, \{h_u : u \in \mathrm{pa}(v)\}\big). \tag{3}$$

This key implements ordinary incremental computation. The evidence layer projects the registered provider snapshot onto the quantities used by the request; two requests can therefore share numerical work while needing different statistical statements.

For a card A, producer p and committed policy π, admission follows

$$\begin{aligned} &\mathrm{ValidCard}(A) \wedge \mathrm{Match}(A, R, \pi) \wedge \mathrm{Cover}(A, R) \\ &\quad \wedge \mathrm{SameProjection}_{D_R}(p, p_A) \wedge U_A \le \delta \Longrightarrow \mathrm{Admissible}(\pi, A, R). \end{aligned} \tag{4}$$

A component revision is first evaluated by an offline regression gate. Common inputs are executed before and after the proposed change; a difference in a quantity declared unaffected vetoes registration. This is a falsification check for the dependency interface. Runtime requests use the registered signatures and do not repeat this image-level audit. The experiment tests three conditional hidden couplings against independent inputs.

Table 2: Typed operating records and their decisions.

Record	Required information	Executable operation
Request/event	Population, service, quantity units and bounds	Compile the failure event
Producer/quantity	Function version, provider dependencies	Compare requested signatures
Source/version	Observation key, sequence, payload integrity	Track the estimate used by the output
Card	Event, producer, population, calibration lineage, n, k, U	Check bound and containment
Plan/record	Request, policy, evidence, active source and history	Issue, withdraw or restore evidence

3.2. Pose refinement and image-to-CAD quality

The image producer receives an identified pose prior and an image observation. Pixel extraction finds fastener centers; the prior supplies coarse CAD-to-pixel predictions for correspondence assignment. Pose refinement uses these matched coordinates and the prior:

$$\begin{aligned}\widehat{\xi} = \arg\min_{\xi} \sum_{i\in\mathcal{I}} \sum_{d\in\{x,y\}} \rho\left(\frac{\pi_\xi(q_i)_d - u_{i,d}}{\sigma_u}\right) \\ + \sum_{d=1}^{3} \left[\rho\left(\frac{\Delta t_d}{\sigma_t}\right) + \rho\left(\frac{\Delta\theta_d}{\sigma_\theta}\right)\right].\end{aligned} \tag{5}$$

Here q_i is a nominal CAD address, u_i a pixel-derived center, and π_ξ the perspective projection. Translation Δt and local rotation $\Delta\theta$ perturb the supplied prior. The componentwise Huber loss has transition 2; σ_u, σ_t and σ_θ scale pixels, translation and rotation. Matching and estimation use the prior and image pixels. Known scene geometry provides the offline quality labels.

Reprojection and spatial mapping measure different properties. Reprojection compares the fitted prediction with the decoder's coordinate. Spatial error evaluates the image-to-CAD map $H_{\widehat{\xi}}$ at a fixed physical image location $u_i^\star$:

$$e_{\mathrm{CAD}} = \max_{i\in\mathcal{P}} \|H_{\widehat{\xi}}(u_i^\star) - q_i^\star\|. \tag{6}$$

Here $\mathcal{P}$ contains every planned physical site. In the new scene population, reprojection is evaluated on every assigned correspondence, including incorrect assignments. Spatial mapping is evaluated at every planned physical site. Ground truth is used only after fitting, for these quality labels and correct-address coverage. This scoring prevents correspondence filtering from hiding a difficult mapping result. A prescribed coordinate-origin intervention is retained separately in the supplementary controlled study.

The execution record identifies the input source, producer, numerical version and digest of its quality-output payload. The registry advances versions monotonically. An attachment must match the active version and payload; rebuilding a plan preserves the source's history. The lifecycle experiment applies these checks to outputs of the same image benchmark.

3.3. Calibration and persistent issuance

For a specified producer and event with k failures among n independent complete calibration inputs, the one-sided bound is

$$U(k,n;\alpha) = \begin{cases} 1, & k = n, \\ \mathrm{Beta}^{-1}(1-\alpha; k+1, n-k), & k < n, \end{cases} \qquad \alpha = \beta/K. \tag{7}$$

The new population uses $\beta = 0.05$ and $K = 6 \times 8 = 48$ producer/event combinations. This family contains six imaging configurations and eight task events; it contains no duplicated localization policies. The union bound gives one simultaneous confidence event for the complete fixed family. Calibration-dependent selection can use any bound in this family; held-out test observations characterize the selected services separately.

Runtime issuance checks source integrity, active version, requested dependency signatures and persistent history. If $A_{s,j}$ is the current attachment condition and $M_{s,j}$ is the source-history state,

$$M_{s,j} = M_{s,j-1} \wedge A_{s,j}, \quad I_{s,j} = M_{s,j} \wedge \text{Admissible}(R), \quad M_{s,0} = \text{true}. \tag{8}$$

Here $A_{s,j}$ checks attachment identity and integrity. Statistical refusal leaves the attachment history intact; an invalid attachment latches it false. A subsequent admissible request can use a valid source, whereas a new plan cannot erase an invalid attempt on that source. An independently registered successor has its own history.

Table 3: Reference contract procedure.

Step	Operation
1	Compile requested quantities, engineering units, bounds and providers.
2	Match producer, population, policy and calibration lineage.
3	Retain a compatible covering card whose bound satisfies the allowance.
4	Construct an exact-event card for a remaining request; refuse if its bound fails.
5	Commit the evidence plan before evaluating the held-out input.
6	Track source versions, output identity and persistent execution history.
7	Issue, withdraw or restore the applicable statement.

4. Evidence transfer through engineering changes

4.1. Requirement containment and provider projection

A requirement specifies acceptable quantity values and the corresponding failure event:

$$S_R = \{z : \ell_{R,j} \le z_j \le u_{R,j},\ j \in J_R\}, \quad E_{p,R} = \{z_p(X) \notin S_R\}. \tag{9}$$

An anchor covers the request when its required quantities include J_R and its intervals are at least as strict. Consequently,

$$S_A \subseteq S_R \Longrightarrow E_{p,R} \subseteq E_{p,A} \Longrightarrow \Pr(E_{p,R}) \le U_{p,A}. \tag{10}$$

The entire containment closure is covered on the same confidence event. A relaxed blur or displacement requirement can therefore use a stronger existing statement.

For quantity j, let D_j contain its providers and $D_R = \bigcup_{j \in J_R} D_j$. The requested signature projects the producer snapshot onto D_R. Transfer after a provider change requires

$$z_{p',j}(X) = z_{p,j}(X) \quad (j \in J_R) \Longrightarrow E_{p',R} = E_{p,R}. \tag{11}$$

In the imaging producer, GSD follows range, focal length and incidence. Blur additionally follows exposure and motion. Coverage, pixel association and refined-pose mapping additionally depend on raster size, illumination and decoding. An ROI or illumination update can therefore retain optical evidence while renewing the affected image-to-CAD statements.

Theorem 1 (Coverage of an authorized event)**.** *For one fixed calibration family, let anchor A cover request R with $U_A \le \delta$. Assume a matched population and invariant requested functions. On the simultaneous confidence event, $\Pr(E_R) \le \delta$. For an issuance indicator I generated by the persistent-history monitor,*

$$\Pr(E_R \cap \{I = 1\}) \le \delta. \tag{12}$$

An invalid attachment prevents subsequent issuance within that source scope.

Proof. Requested-function identity preserves the request event. Containment gives $E_R \subseteq E_A$, and simultaneous calibration gives $\Pr(E_A) \le U_A \le \delta$. Intersecting with issuance preserves the bound. The conjunction in Eq. (8) remains false after an invalid attachment. □

4.2. Reuse with exact-event fallback

For a committed producer/policy, let $\mathcal{A}_\pi(R)$ contain compatible covering anchors with $U_A \le \delta$, and $\mathcal{D}_\pi(R)$ contain compatible exact-event cards with $U_D \le \delta$. The route is

$$Q_\pi(R) = \begin{cases} \text{reuse}(A), & \mathcal{A}_\pi(R) \neq \varnothing, \\ \text{direct}(D), & \mathcal{A}_\pi(R) = \varnothing,\ \mathcal{D}_\pi(R) \neq \varnothing, \\ \text{refuse}, & \text{otherwise.} \end{cases} \tag{13}$$

Both branches use the same fixed confidence allocation. An exact-event fallback preserves support when the available anchor is too strict. The algorithm separates this request decision from the producer computation required to renew its quantities.

Proposition 2 (Support preservation with exact fallback)**.** *Suppose every compared card uses the same n calibration inputs and one-sided error allocation α. For a transferred request, its quantities agree on those inputs. If the complete exact-event catalogue admits R, reuse followed by exact fallback admits R. Conversely, admission by a covering anchor implies admission by that exact-event catalogue.*

Proof. If a covering anchor admits the request, containment and input-wise quantity agreement give $k_R \le k_A$. The binomial upper bound is nondecreasing in its failure count at fixed n and α, so $U_R \le U_A \le \delta$. If no anchor admits the request, the fallback evaluates its exact event with the same inputs and allocation. Both procedures therefore give the same support decision. □

The result separates support from preparation work. A stricter anchor may fail while a relaxed exact event passes; its fallback is then essential. An implementation that stops after the failed anchor loses support. Confidence allocations are kept identical across methods so that this effect is not mixed with a change in multiplicity penalty.

4.3. Matched comparison methods

Whole renewal evaluates the full image producer and constructs exact-event cards. The DAG baseline reuses cached stages and exact cards whose projected signatures agree. The contract uses the same DAG and adds containment with exact fallback. An independent forward-chaining implementation tests bound, interval and provider facts before applying the same fallback. All constructed cards count toward preparation cost, including cards supporting a refusal. No method changes the numerical estimator or the allocation α.

5. Experimental design

5.1. Diverse aircraft-skin simulation

The main experiment draws 256 calibration inputs and 256 held-out inputs from a mixture of four layouts: parallel joint rows, curved joint rows, staggered fasteners and an access-panel perimeter. Each scene independently draws planar, single-curvature or double-curvature geometry, camera pose and surface appearance. Layouts contain 28–36 planned sites with variable head radii and local position offsets. The sampling unit is a complete scene; paired producer configurations share that scene.

Missing fasteners occur with probability 0.025 per site. An opaque edge region covers a uniformly sampled 0–10% of the raster area. The physical sites differ from their nominal CAD addresses through a shared 0.4-mm displacement standard deviation and a local 0.15-mm standard deviation. Painted and metallic appearances use sampled roughness, view-dependent highlights, joints, stains, scratches and intensity noise. The renderer uses perspective ray/surface intersection. These defined variations make correspondence assignment and mapping quality observable under imperfect input conditions.

Baseline resolution is 768×576 pixels; the expanded region is 896×672. Focal length is 1,200–1,600 pixels, camera distance 0.70–1.15 m, exposure 3–7 ms and motion 0.06–0.18 m/s. Five changes independently increase focal length by 15%, exposure by 50%, expand the raster, multiply illumination by 0.45, or lower

the decoder contrast threshold from 18 to 10. Thus 512 scenes produce 3,072 paired producer outputs. The decoder change modifies actual feature extraction, rather than prescribing a coordinate displacement.

Pixel extraction uses local contrast and connected components of 5–240 pixels. The pose prior gates minimum-cost CAD assignment at 30 pixels. At least six matches trigger the robust fit in Eq. (5), with 0.5-pixel, 3-mm and 0.001-rad scales. The estimator never receives physical-site positions. The first held-out input in each layout family and a separate stress input are shown in Fig. 3.

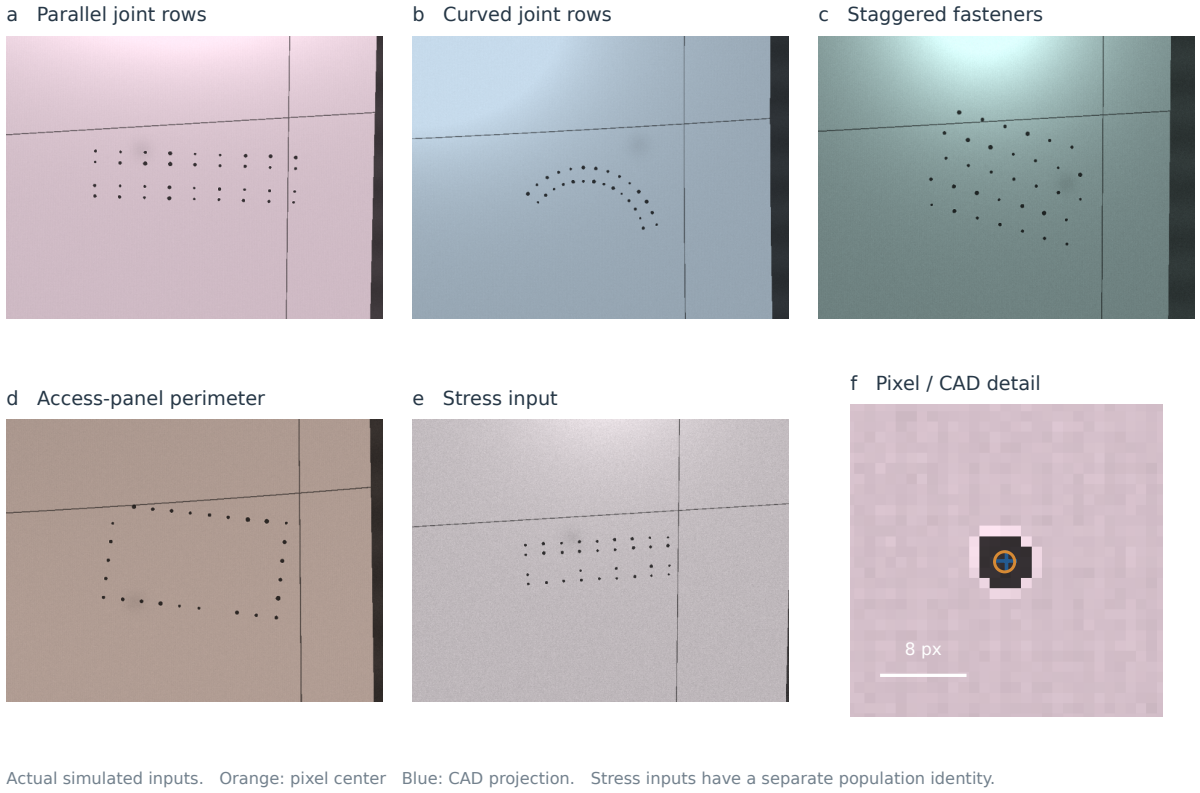


Figure 3: Executed image inputs and correspondences. The first held-out scene in each layout family illustrates sampled layout and appearance; the remaining panels show the first stress input and an enlarged decoded/CAD correspondence. Orange marks pixel-derived centers; blue marks CAD projections. All images are actual producer outputs.

5.2. Requests and independent evaluation

The eight request definitions are retained from the controlled surface study, before sampling the new population. Optical requests constrain sampling and blur; acquisition adds correct-address coverage; association constrains coverage, reprojection and spatial mapping; complete service combines these quantities. Strict and relaxed forms give eight events per producer. Thresholds and producer configurations remain fixed across the calibration and test sets. Full values, card counts and seed lists are provided in the Supplementary Material.

A separate 128-scene stress population increases missing-site probability to 0.14, occluded area to 0–35%, shared CAD displacement to 2 mm and local displacement to 0.6 mm. It tests the operating envelope, with its own population identity. These outcomes are reported separately from the calibrated population. Earlier regular-grid and scan-registration studies are retained as controlled mechanism and implementation comparisons in the supplement.

5.3. Measured source renewal and evidence preparation

The performance experiment uses 16 independent timing scenes, five updates and eight requests: 640 paired input/update/request cases per method. A baseline stage cache is supplied equally before each case. Whole renewal executes the full producer; the other methods evaluate the requested stages through the same DAG. Every method is executed, with rotated order, and its requested numerical quantities are checked against the others. Timing includes cache lookup, required numerical stages and a scalar event check. Process CPU and elapsed time are recorded separately.

Catalogue construction is timed separately on all 256 stored calibration records. Eleven repetitions measure event evaluation, bound inversion and routing. This boundary isolates evidence-processing cost from image computation. A dependency-only baseline and the contract have the same numerical-stage capabilities; their difference is the event catalogue constructed above those stages.

Execution uses Python 3.12 on Linux with an AMD EPYC 9V74 host. Numerical workers use one thread; the timing stream uses one worker, while population generation uses two. Per-case timings, stage counts and implementation versions are included with the source package.

5.4. Component registration and complete source traces

The offline regression experiment uses 64 further independent inputs. Three paired faults conditionally couple ROI to focal length, illumination to exposure, or decoder configuration to focal length. The triggering conditions are camera-side, painted appearance and curvature family, respectively. The same inputs also execute each clean declared update. A registration veto requires a numerical difference in a quantity declared unaffected; no decision uses a change-table oracle as numerical ground truth.

The lifecycle experiment then uses every held-out image record and all eight requests. A trace issues the baseline statement, evaluates an exposure update, restores the original producer and tests independent successor recovery. Separate fault scopes inject a stale version, altered output payload and wrong producer, each followed by a reconstructed-plan retry. Three ablations remove version/integrity checks, persistent history, or payload integrity. These deterministic traces measure protocol behavior on computed outputs; scene-level image-quality evaluation supplies the statistical evidence.

6. Results

6.1. Image quality and calibrated support

The held-out baseline images have mean correct-address coverage 96.14%. Pixel-based refinement reduces median prior translation error from 4.328 to 1.149 mm. Median reprojection is 0.639 pixel; median and 95th-percentile mapping errors are 0.502 and 1.052 mm. The complete per-configuration distributions and scene-level results are archived with the study.

At allowance 0.15, the fresh 48-event catalogue admits 41 contexts (Fig. 4). Increased exposure refuses strict acquisition and complete service. Lower illumination additionally removes strict association and the relaxed association/complete requests. Each complete renewal method gives the same admission pattern. These refusals follow the newly executed image quantities and calibration counts.

6.2. Evidence preparation and fallback

Whole renewal constructs 48 event cards; the dependency-aware exact cache constructs 42; the contract and independent rules implementation each construct 26. Their record evaluations are 12,288, 10,752 and 6,656, respectively. The reduction relative to the DAG baseline is 38.10%, including refusal decisions and failed anchors. Anchor-only routing admits 38 contexts. Exact fallback recovers three: relaxed acquisition and complete service after the exposure update, and relaxed acquisition under lower illumination. Thus the complete route retains all 41 directly supportable contexts (Fig. 5).

The compiled contract takes 2.725 ms for this catalogue, compared with 4.290 ms for exact caching. The absolute saving is 1.565 ms. Event containment therefore reduces evidence-processing work at this scale; expensive image-stage reuse is quantified separately below. The rule implementation agrees on every decision and constructed-card count, while its interpreted predicates have a different preparation cost.

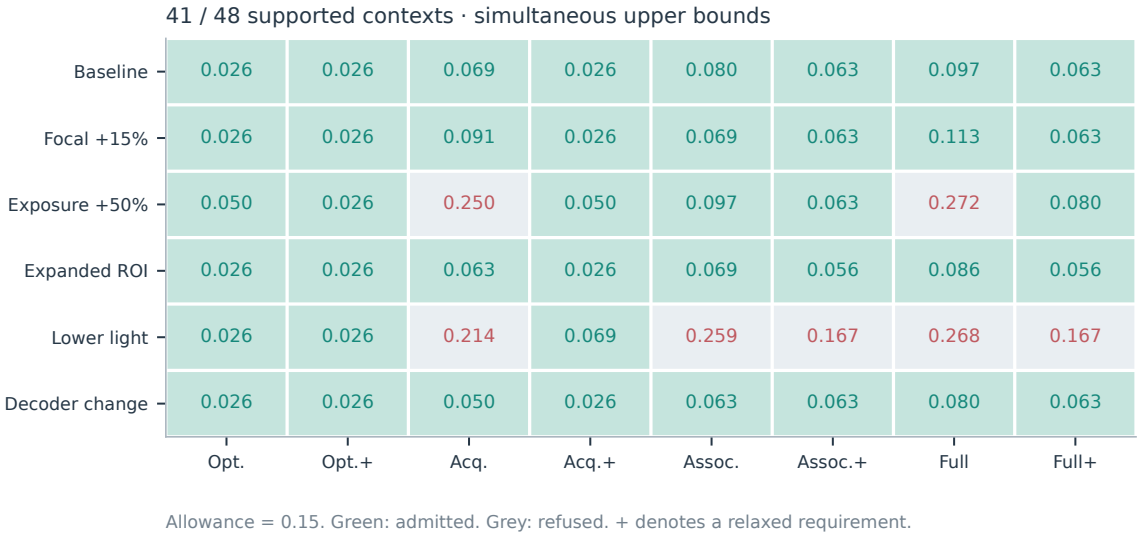


Figure 4: All simultaneous risk bounds for the new population. Every method uses the same 256 calibration inputs, allocation 0.05/48 and allowance 0.15. Grey cells are refused.

Table 4: Complete evidence-catalogue construction. Metadata projections are compiled before timing; all methods share the same stored quantities and allocation.

Method	Supported	Cards	Record evaluations	CPU (ms)
Whole renewal	41/48	48	12,288	4.419
DAG exact cache	41/48	42	10,752	4.290
Contract + fallback	41/48	26	6,656	2.725
Rules + containment	41/48	26	6,656	4.777

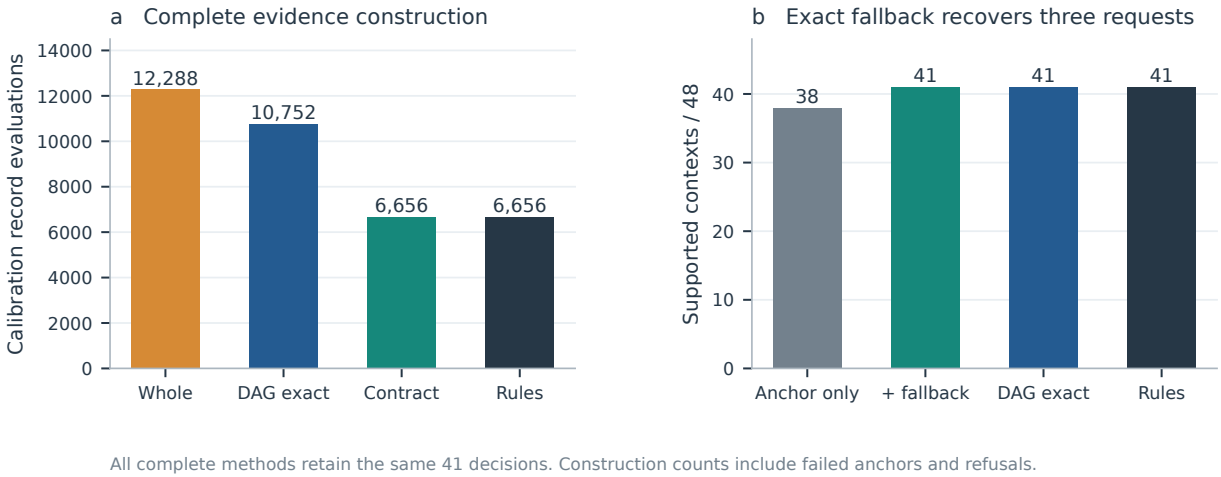


Figure 5: Evidence work and support attribution. The full route restores the three contexts lost by anchor-only reuse. The independent rules implementation has the same evidence semantics and support.

6.3. Measured numerical renewal

Across 640 paired input/update/request cases, whole renewal consumes 183.993 s of process CPU and the DAG exact cache consumes 80.486 s, a reduction of 56.26%. A paired bootstrap over the 16 timing inputs gives a 95% interval of 55.87–56.65% for that CPU reduction. Contract and rules paths consume 80.279 and 79.757 s (Fig. 6). All requested numerical outputs agree across methods.

The three DAG-based methods execute identical stage counts: 192 geometry, 288 radiance, 384 sensor, 480 decode, 480 pose and 192 explicit optical nodes. Whole renewal executes 640 of each node. Focal and ROI updates retain less image work than exposure and decoder updates. These measured reductions belong to dependency-directed source renewal shared by the three methods. Containment's additional effect is the smaller evidence catalogue in Table 4.

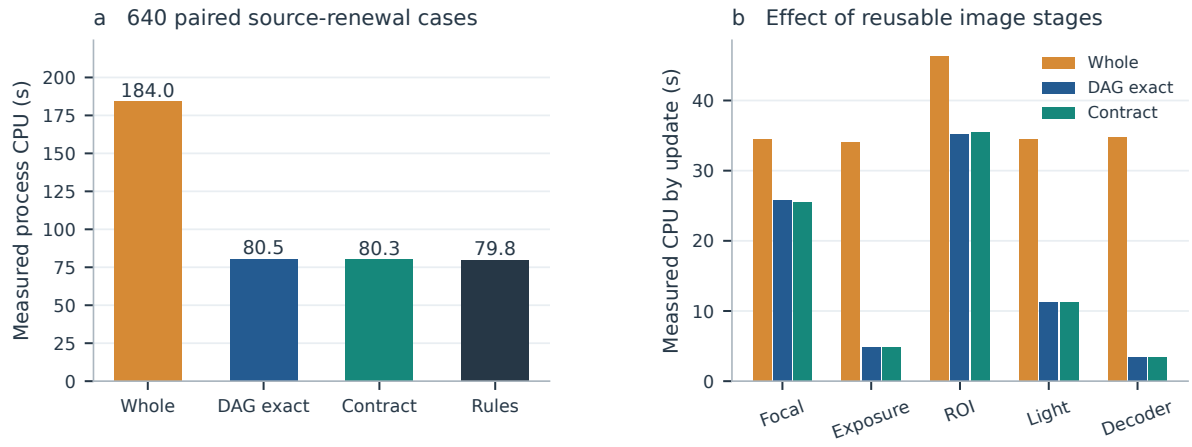


Figure 6: Executed source-renewal cost with a common baseline cache. Every method runs every case; plotted values are sums of measured process CPU, rather than estimates from logical call counts. The update breakdown exposes which stages can be retained.

6.4. Component registration and source lifetime

The 64 independent regression inputs activate 86 of 192 conditional hidden-coupling cases. Every activated case changes an unaffected quantity and triggers registration veto; all 192 clean update pairs pass. The activation counts are 24 for ROI/focal, 36 for illumination/exposure and 26 for decoder/focal (Fig. 7). The check directly compares recomputed quantities, complementing the declared dependency interface.

The lifecycle study uses 2,048 held-out source/request contexts. The contract agrees with the specified issuance decision throughout baseline, producer change, restoration and independent-successor stages. It refuses stale versions, altered payloads and mismatched producers, and preserves that refusal after plan reconstruction within each fault scope. Removing history permits 6,144 invalid retries; removing version/integrity checks permits 10,240 invalid issuances; removing payload integrity permits 4,096. Figure 8 separates normal operation from those conformance ablations.

6.5. Operating envelope under stronger disturbances

The separate stress population reduces mean coverage from 96.14% to 72.69% and increases the 95th-percentile mapping error from 1.052 to 13.095 mm. Strict complete-service failures increase from 4/256 in the nominal held-out population to 77/128 in stress (Fig. 9). Optical sampling alone remains acceptable, whereas missing/occluded correspondences and CAD discrepancy degrade the image-to-model task. This separation identifies the population conditions associated with the calibrated statement.

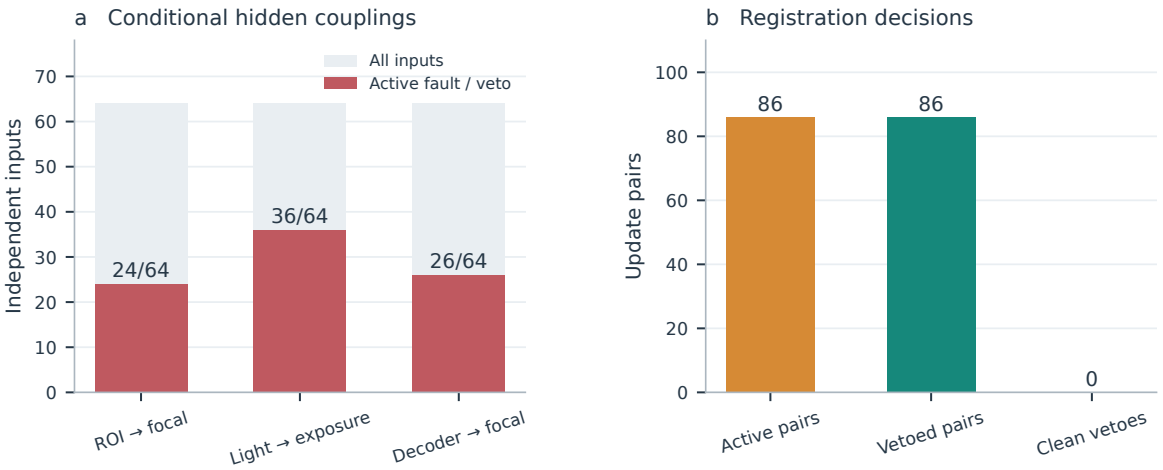


Figure 7: Numerical regression at component registration. Vetoes follow recomputed quantities on common inputs. Conditional branches supply both active and inactive examples within each fault class.

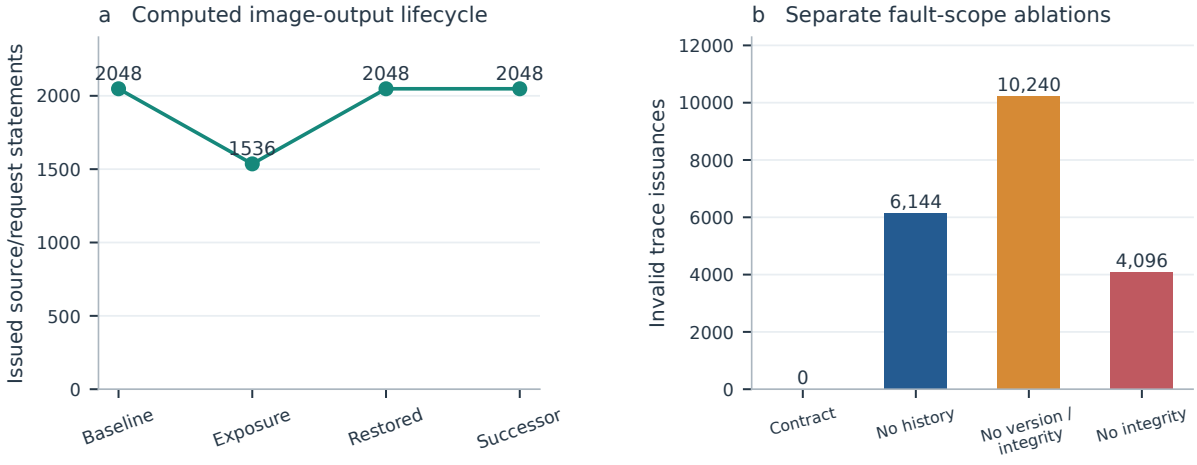
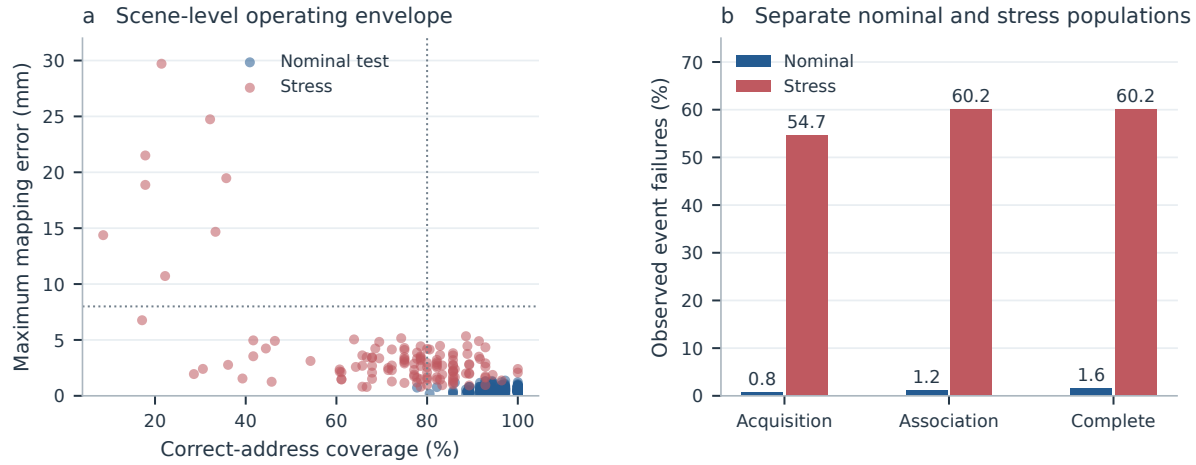


Figure 8: Statistical evidence and attachment history in the same image-output workflow. Producer restoration recovers compatible statements. Invalid attachment history persists within its source scope; a separately registered successor can issue again. Fault traces are deterministic conformance evaluations.



Nominal: 256 held-out scenes. Stress: 128 further scenes with more occlusion, missing sites and CAD mismatch.

Figure 9: Separate scene-level nominal and stress outcomes. Each point is one complete input. Dashed thresholds illustrate strict coverage and mapping requirements; complete service additionally constrains sampling, blur and reprojection.

7. Discussion

The experiments separate two reuse operations. Stage fingerprints retain numerical values when their implemented inputs are unchanged. Event containment retains a statistical statement when its failure event covers the new request. The dependency cache therefore supplies the expensive image-computation benefit, while the contract reduces repeated evidence construction above that cache. The independent rule implementation recovers the same decisions, locating the contribution in the composed evidence semantics and their traceable execution.

The support-preservation result explains why reuse need not trade away usable requests. A failed strict anchor triggers an exact-event evaluation, and all methods retain the same confidence allocation. The observed equality of supported contexts is thus an implementation check of a stated property. Work counts include refused requests and failed anchors, so the preparation comparison accounts for the complete decision process.

Engineering changes and difficult inputs demand different responses. A registered component update changes producer identity and can require selective renewal. Occlusion or CAD discrepancy changes the image-quality distribution and is evaluated against its specified population. The separate stress results expose this operating boundary directly. A current residual, a cached computation and a calibrated population statement answer different questions; the evidence interface records the relations needed to use them together.

The source traces show why restoration is also a stateful operation. Restoring a producer can recover its compatible statistical statement. It cannot erase an invalid attachment already recorded for the same source. A successor source starts a new scope. This distinction connects evidence reuse to the output's lifetime rather than to a single successful function call.

8. Conclusion

Executable evidence contracts join requirement semantics, producer dependencies and output history in one inspection workflow. With aligned calibration inputs and confidence allocation, containment with exact fallback preserves the support of direct renewal. The new image study retains all 41 supported contexts while reducing evidence-record evaluations by 38.10%. Measured numerical renewal confirms the benefit of

conventional stage caching shared by the complete methods, while component regression and source traces establish the distinct roles of dependency validity and persistent attachment history. The framework provides an executable basis for retaining, renewing and issuing engineering evidence after change.

References

[1] R. Wang, A. B. Nellippallil, G. Wang, Y. Yan, J. K. Allen, F. Mistree, A process knowledge representation approach for decision support in design of complex engineered systems, Advanced Engineering Informatics 48 (2021) 101257. doi:10.1016/j.aei.2021.101257.

[2] J. Hao, L. Zhao, J. Milisavljevic-Syed, Z. Ming, Integrating and navigating engineering design decision-related knowledge using decision knowledge graph, Advanced Engineering Informatics 50 (2021) 101366. doi:10.1016/j.aei.2021.101366.

[3] H. Wang, V. Thomson, C. Tang, Change propagation analysis for system modeling using semantic web technology, Advanced Engineering Informatics 35 (2018) 17–29. doi:10.1016/j.aei.2017.11.004.

[4] A. Mokhov, N. Mitchell, S. Peyton Jones, Build systems à la carte, Proceedings of the ACM on Programming Languages 2 (ICFP) (2018) 1–29. doi:10.1145/3236774.

[5] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, L. Lei, Learn then test: Calibrating predictive algorithms to achieve risk control, The Annals of Applied Statistics 19 (2) (2025) 1641–1662. doi:10.1214/24-AOAS1998.

[6] U. Mehmood, S. Sheikhi, S. Bak, S. A. Smolka, S. D. Stoller, The black-box simplex architecture for runtime assurance of autonomous CPS, in: NASA Formal Methods, Vol. 13260 of Lecture Notes in Computer Science, Springer, 2022, pp. 231–250. doi:10.1007/978-3-031-06773-0_12.

[7] L. Petnga, M. Austin, An ontological framework for knowledge modeling and decision support in cyber-physical systems, Advanced Engineering Informatics 30 (1) (2016) 77–94. doi:10.1016/j.aei.2015.12.003.

[8] L. Ma, T. Hartmann, A proposed ontology to support the hardware design of building inspection robot systems, Advanced Engineering Informatics 55 (2023) 101851. doi:10.1016/j.aei.2022.101851.

[9] C. Quinn, J. McArthur, A case study comparing the completeness and expressiveness of two industry recognized ontologies, Advanced Engineering Informatics 47 (2021) 101233. doi:10.1016/j.aei.2020.101233.

[10] J. Xiao, Q. Wang, Y. Li, Z. Yu, H. Zhou, B. Lin, A fully automated dm-bim-bem pipeline enabling graph-based intelligence, interoperability, and performance-driven early design, Advanced Engineering Informatics 75 (2026) 104824. doi:10.1016/j.aei.2026.104824.

[11] P. Besl, N. D. McKay, A method for registration of 3-D shapes, IEEE Transactions on Pattern Analysis and Machine Intelligence 14 (2) (1992) 239–256. doi:10.1109/34.121791.

[12] T. D. Barfoot, P. T. Furgale, Associating Uncertainty With Three-Dimensional Poses for Use in Estimation Problems, IEEE Transactions on Robotics 30 (3) (2014) 679–693. doi:10.1109/tro.2014.2298059.

[13] A. Censi, An accurate closed-form estimate of ICP's covariance, in: Proceedings 2007 IEEE International Conference on Robotics and Automation, 2007, pp. 3167–3172. doi:10.1109/robot.2007.363961.

[14] S. Bonnabel, M. Barczyk, F. Goulette, On the covariance of ICP-based scan-matching techniques, in: 2016 American Control Conference (ACC), 2016, pp. 5498–5503. doi:10.1109/acc.2016.7526532.

[15] M. Brossard, S. Bonnabel, A. Barrau, A New Approach to 3D ICP Covariance Estimation, IEEE Robotics and Automation Letters 5 (2) (2020) 744–751. doi:10.1109/lra.2020.2965391.

[16] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, K. G. Larsen, Contracts for system design, Foundations and Trends in Electronic Design Automation 12 (2–3) (2018) 124–400. doi:10.1561/1000000053.

[17] C. J. Clopper, E. S. Pearson, The use of confidence or fiducial limits illustrated in the case of the binomial, Biometrika 26 (4) (1934) 404–413. doi:10.1093/biomet/26.4.404.

[18] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster, Conformal risk control, in: The Twelfth International Conference on Learning Representations, 2024.
URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f3549ef9b5ff520a7e41ff3cc306ab2b-Abstract-Conference.html